

transpareo-md2pdf

Markdown to PDF, rendered through headless Chromium. Tables, code blocks and CSS behave the way they do on the web, because a browser is doing the layout.

Contents

Why a browser	3
Install	3
Usage	4
As a library	4
What it understands	4
Images	5
Table of contents	5
Title page	6
Footnotes	6
Options reference	6
Commands	7
Content	7
Typography	8
Layout	8
Branding	9
Style	9
Misc	9
Configuration	10
Using your own logo	11
Custom localizations	12
Environment	12
Docker	12
Continuous integration	13
Contributing	13
License	13

Why a browser

Every layout problem a document generator has to solve has already been solved, repeatedly, by browser engines: column sizing, orphan and widow control, page breaks inside tables, font fallback, ligatures, bidirectional text. Rather than reimplement any of it, md2pdf hands Chromium a self-contained HTML document and asks it to print.

	md2pdf	wkhtmltopdf	pandoc --pdf
Layout engine	current Chromium	WebKit, ~2012	LaTeX
CSS support	whatever Chrome does	partial, dated	none
TOC page numbers	read from the PDF	manual	yes
External programs	Chromium	wkhtmltopdf	pandoc + TeX
Install size	browser only	~50 MB	~2 GB with TeX

Chromium is the only external program required. Markdown parsing, syntax highlighting and PDF inspection are gems, so `bundle install` covers them.

Install

```
gem install transpareo-md2pdf
md2pdf doctor
```

```
ok    chromium      148.0.7778.215  /usr/bin/chromium
ok    commonmarker    2.9.0          gem
ok    nokogiri         1.19.4         gem
ok    rouge            4.7.0          gem
ok    pdf-reader       2.15.1         gem
ok    rubyzip          2.4.1          gem
```

Most machines already have a browser. If yours does not, either install one with your package manager or let md2pdf fetch a known-good build:

```
md2pdf install-deps
```

That downloads Google's `chrome-headless-shell` for your platform into `~/.local/share/md2pdf`, verifies it against a SHA-256 sum shipped in the gem, and unpacks it. No root, no `PATH` changes, and it never runs on its own. `--force` reinstalls; `--latest` takes current stable instead of the pinned build, which skips checksum verification since there is nothing pinned to compare against.

Chromium is resolved from `CHROMIUM`, then the `install-deps` directory, then `PATH`, then the standard macOS app bundles.

Usage

```
md2pdf file.md
md2pdf *.md           # all .md in current dir
md2pdf 'docs/*.md'    # globs are expanded internally
md2pdf                # same as md2pdf *.md
md2pdf --flat file.md  # only H1 rendered as heading
md2pdf --unwrap file.md # join hard-wrapped paragraphs
md2pdf --no-toc file.md # skip the TOC
```

PDFs are written next to their inputs unless `--output` or `--output-dir` says otherwise.

As a library

```
require 'transpareo/md2pdf'

Transpareo::Md2pdf.convert('report.md', toc: true, locale: 'de')
```

Returns true when the PDF was written. Missing dependencies raise `MissingDependencyError`, render failures raise `ConversionError`, and nothing in the library calls `exit`.

What it understands

Everything in GitHub Flavored Markdown, plus a few things beyond it.

Feature	Notes
Tables	Header words wrap individually so columns size sanely
Fenced code	Highlighted by Rouge, inline styles, self-contained

Feature	Notes
Images	Local files inlined, so the PDF has no external refs
Footnotes	Renumbered in reference order, duplicates merged
Task lists	- [] and - [x]
Strikethrough	~text~
Autolinks	Bare URLs become links
Fenced divs	::: intro blocks, for title-page control
Front matter	An md2pdf: YAML block, stripped before rendering

Images

Paths are resolved **relative to the markdown file**, not to your working directory, so a document renders the same wherever you run md2pdf from. Local images are read and embedded as data URIs, which means the finished PDF carries its own artwork and does not break when the source tree moves.

PNG, JPEG, GIF, SVG, WebP, AVIF, BMP and ICO are recognised. Remote `http(s)` sources are left for the browser to fetch, so they need network access at render time. A missing or unreadable image warns and is skipped rather than failing the whole document.

Table of contents

A TOC covering H2 and H3 is inserted by default on its own page, between the title page and the first H2, carrying **real page numbers**. It is skipped for short documents: both `--toc-min` (default 3 H2s) and `--toc-min-words` (default 1500) must be met.

The numbers are not estimated. The document is rendered once, Chromium writes a PDF destination for every heading the TOC links to, and the second pass reads that table back and bakes the numbers in. Because they come from the PDF's own structure rather than from scraping extracted text, they stay correct even when a heading lands right at a page boundary. This roughly doubles render time.

Title page

The H1 and its lead block are vertically centered on page 1 **only when the document has a TOC and no** `::: intro :::` **block**. Otherwise the title flows from the top of page 1.

Use `::: intro :::` to keep an introduction on page 1, right after the title:

```
# Document Title
```

```
*Subtitle line.*
```

```
::: intro
```

```
A multi-paragraph introduction that lands on page 1, right after  
the subtitle, before the TOC starts on the following page.
```

```
:::
```

```
## First Section
```

Footnotes

```
The first attempt failed.[^attempt]  
A later attempt succeeded.[^attempt]
```

```
[^attempt]: Internal lab note, 2026-04-19.
```

Each `[^id]` becomes a numbered superscript link, numbered in the order references appear. Entries whose definition text is identical are merged into one item, which is what you want when the same source is cited from several places. Every entry links back to its first reference.

A trailing `## Footnotes` heading in the source is consumed so it does not double up with the rendered one. The heading is set with `--footnotes-label="Sources"`, or omitted entirely with `--footnotes-label=""`.

Options reference

Every flag, with the config key that sets the same thing. Flags beat front matter, front matter beats `.md2pdf.yml`.

Commands

Command	Description
<code>doctor</code>	Report dependency status and exit non-zero if any are missing
<code>install-deps</code>	Download a pinned, checksum-verified Chromium
<code>install-deps --latest</code>	Take current stable instead of the pinned build; skips checksum verification
<code>install-deps --force</code>	Reinstall even if that version is already present

Content

Flag	Config key	Default	Description
<code>--flat</code>	<code>flat</code>	<code>false</code>	Demote H2/H3 to bold paragraphs, leaving H1 the only heading
<code>--single-heading</code>	<code>flat</code>	<code>false</code>	Alias for <code>--flat</code>
<code>--unwrap</code>	<code>unwrap</code>	<code>false</code>	Join hard-wrapped paragraph lines back into one
<code>--toc</code>	<code>toc</code>	<code>on</code>	Force a TOC, ignoring the auto-skip thresholds below
<code>--no-toc</code>	<code>toc: false</code>		Disable the TOC entirely
<code>--toc-depth=N</code>	<code>toc-depth</code>	<code>2</code>	<code>1</code> = H2, <code>2</code> = H2 + H3, <code>3</code> = also H4
<code>--toc-label=TEXT</code>	<code>toc-label</code>	per locale	TOC heading text
<code>--toc-min=N</code>	<code>toc-min</code>	<code>3</code>	H2s required before a TOC appears
<code>--toc-min-words=N</code>	<code>toc-min-words</code>	<code>1500</code>	Words required before a TOC appears
<code>--footnotes-label=TEXT</code>	<code>footnotes-label</code>	per locale	Footnotes heading; <code>""</code> renders the list with no heading
<code>--locale=CODE</code>	<code>locale</code>	<code>auto</code>	Sets default labels and the <code>lang</code> attribute

Flag	Config key	Default	Description
	locales		Custom label sets, see below. Config only

Both `--toc-min` and `--toc-min-words` must be satisfied for a TOC to appear automatically. `--toc` overrides both.

Typography

Flag	Config key	Default
<code>--font-size=Npt</code>	font-size	11pt
<code>--line-height=N</code>	line-height	1.8
<code>--font-family="..."</code>	font-family	Plus Jakarta Sans, DejaVu Sans, Helvetica
<code>--code-font-family="..."</code>	code-font-family	JetBrains Mono, DejaVu Sans Mono, Menlo

Fonts must be installed on the rendering machine. Chromium falls back silently if one is missing, so check the output when using a font the machine may not have.

Layout

Flag	Config key	Default
<code>--page-size=NAME</code>	page-size	A4 . Any CSS page size: Letter , A5 , Legal
<code>--margins="T R B L"</code>	margins	22mm 20mm 24mm 20mm , in CSS order
<code>--output=FILE</code>	output	<input>.pdf
<code>--output-dir=DIR</code>	output-dir	alongside the input

A relative `output-dir` in a config file resolves against that config file, so `output-dir: build` collects PDFs in the project's `build/` no matter which directory you run from. As a flag it resolves against your shell. `md2pdf` prints the absolute path of each file it writes.

Branding

Flag	Config key	Default	Description
<code>--logo=PATH</code>	<code>logo</code>	none	SVG logo. The footer version is derived from the same file in grey, so one asset covers both
<code>--no-header-logo</code>	<code>header-logo:</code> <code>false</code>	shown	Hide the first-page logo
<code>--no-footer-logo</code>	<code>footer-logo:</code> <code>false</code>	shown	Hide the per-page footer logo
<code>--no-page-numbers</code>	<code>page-numbers:</code> <code>false</code>	shown	Hide the page counter

md2pdf ships no logo and renders unbranded out of the box. The Transpareo mark on the sample pages above is exactly that: a sample, supplied to the renderer the same way you would supply your own. It lives in this repository for the demo and is deliberately excluded from the published gem, so installing md2pdf gives you a blank canvas rather than somebody else's branding.

See [Using your own logo](#) below.

Style

Flag	Config key	Default	Description
<code>--link-color=#hex</code>	<code>link-color</code>	<code>#0a4a90</code>	Anchor colour throughout
<code>--custom-css=FILE</code>	<code>custom-css</code>	none	Appended to the stylesheet, so it wins on ties

Misc

Flag	Description
<code>--open</code>	Open the PDF with <code>xdg-open</code> afterwards. Rejected when more than one file would be converted

Flag	Description
<code>-v</code> , <code>--version</code>	Print the version and exit
<code>-h</code> , <code>--help</code>	Print help and exit

Exit codes: `0` success, `1` a render or dependency failure, `2` a usage error, `130` interrupted.

Configuration

Settings come from three places, highest priority first.

1. CLI flags.

2. YAML front matter, under an `md2pdf:` key so it cannot collide with anything else:

```

---
md2pdf:
  font-size: 14pt
  toc-label: Inhalt
---

# My Document

```

The block is stripped before rendering, so it never appears in the PDF.

3. `.md2pdf.yml`, found by walking up from the input file until one is found or `$HOME` is reached:

```

font-size: 14pt
line-height: 1.6
page-size: A4
margins: "22mm 20mm 24mm 20mm"
logo: assets/brand/logo.svg
link-color: "#0a4a90"
page-numbers: false

```

Keys mirror the CLI flags without the `--` prefix. Negating flags invert: `--no-page-numbers` becomes `page-numbers: false`.

Because md2pdf walks up from the document until it finds a config, one file at the root of a docs tree brands and styles everything beneath it. Committing `.md2pdf.yml` next to your docs means every contributor produces identical PDFs without passing any flags.

Using your own logo

Point `--logo` at an SVG, or set `logo:` in `.md2pdf.yml` so every document under that directory picks it up:

```
logo: assets/brand/logo.svg
```

A relative path is relative to whatever declared it. A path in `.md2pdf.yml` resolves against that config file, one in a document's front matter resolves against the document, and one given as a flag resolves against your shell, which is what typing a path into a terminal implies. The rule covers every setting naming a file or directory: `logo`, `custom-css`, `output` and `output-dir`.

That means a repository can commit its branding next to its docs and every contributor produces identically branded PDFs from any working directory, without passing a flag.

`MD2PDF_LOGO` sets a per-machine default when the asset lives outside the project.

One asset covers both placements. The header logo on page 1 uses your original colours. The small footer logo repeated on every page is derived from the same file by rewriting each fill, gradient stop and inline `fill:` to grey, so there is no second file to keep in sync.

What makes a good logo file here:

- **SVG.** It is the only format the recolouring understands, and it stays sharp at any zoom. Raster formats work in document body images, not as the logo.
- **A wide wordmark.** The header box is 42mm by 6.95mm and the footer is 18mm by 2.98mm, both roughly 6:1. A square or tall mark will be squashed, so crop the artwork or set your own sizes with `--custom-css`.
- **Colour defined as fills or gradient stops**, not embedded raster or CSS classes, or the grey footer derivation will not find anything to recolour.

If the file is missing, md2pdf warns and renders without it rather than failing the document. Use `--no-header-logo` or `--no-footer-logo` to suppress either placement.

The sample logo in this repository is Transpareo's trademark and is present only to demonstrate the feature. The MIT licence covers the code, not the brand assets. Replace it with your own.

Custom localizations

The built-in label sets cover `en`, `de`, `fr`, `es`, `it`, `pt` and `nl`. A `locales:` block overrides any of them and adds locales that are not built in:

```
logo: assets/brand/logo.svg

locales:
  de:
    toc-label: Inhaltsverzeichnis # override a built-in
  sv:
    toc-label: Innehåll           # add a new one
    footnotes-label: Fotnoter
```

An entry only has to name the labels it changes; anything omitted keeps its built-in wording. Once a locale is defined here, filename detection recognises it too, so `handbok.sv.md` picks up the Swedish labels and gets `lang="sv"` on the output.

The document itself still wins, so a single file can opt out:

```
---
md2pdf:
  toc-label: Agenda
---
```

Environment

<code>CHROMIUM=/usr/bin/chromium</code>	Browser override
<code>MD2PDF_LOGO=/path/to/logo.svg</code>	Default for <code>--logo</code>
<code>MD2PDF_HOME=~/.local/share/md2pdf</code>	Managed install directory

Docker

```
docker build -t transpareo-md2pdf .
docker run --rm -v "$PWD:/work" transpareo-md2pdf report.md
```

The image carries Chromium and the fonts it needs.

Continuous integration

GitHub's Ubuntu runners already ship Chrome, which md2pdf finds on `PATH`, so usually nothing extra is needed:

```
- uses: ruby/setup-ruby@v1
  with:
    ruby-version: '3.3'
    bundler-cache: true

- run: bundle exec md2pdf docs/*.md
```

Avoid `apt-get install chromium-browser` on Ubuntu runners: it is a snap transitional package that does not run headless. On images with no browser at all, `bundle exec md2pdf install-deps` caches well, since the download URL is pinned.

Contributing

See [CONTRIBUTING.md](#) for the architecture, the filter pipeline and the release process.

License

MIT. See [LICENSE.txt](#).

The licence covers the source. `docs/assets/transpareo-logo.svg` is a trademark of Transpareo, included solely as a worked example of the branding options, and is not part of the distributed gem.